ForgeOS
Reference Manual

End-to-end workflow, REST API reference, and the full test catalog for the AI demand-to-delivery platform.

Table of Contents

ForgeOS — System Overview & End-to-End Workflow

What ForgeOS Is

ForgeOS is a unified, AI-assisted demand-to-delivery operating system. A client submits a free-text demand; an AI delivery architect refines it through a multi-turn conversation; the platform produces a detailed plan, allocates a team (human + AI agents), routes it to a manager for review and approval, executes the build through an agent fleet, and tracks delivery end to end with full audit, SWON/WON commercial records, notifications, and a sanitized higher-management portfolio.

The platform is multi-tenant: every business row carries a `tenant_id` for isolation. Access is governed by a role hierarchy enforced both in the backend (RBAC dependencies) and the frontend (route guards + role-tailored navigation).

Role Hierarchy

ForgeOS defines ten roles with an explicit hierarchy level. Higher levels can see more; lower levels see a focused, role-specific workspace.

Role	Level	Workspace landing	Primary responsibility
Executive	6	/dashboard/executive	Org-wide KPIs and trends
Higher Manager	5	/dashboard/higher-manager	Sanitized portfolio oversight
Manager	4	/dashboard/manager	Plan review, approval, team edits
Middleware	3	/dashboard/middleware	Intake routing and handoffs
Leader	2	/dashboard/leader	Squad execution and task board
Delivery Team	2	/dashboard/delivery	Squad throughput
Member	1	/dashboard/member	Individual task execution
Contributor	1	/dashboard/contributor	Tasks plus code commits
Viewer	0	/dashboard/viewer	Read-only portfolio and audit
Client	0	/client	Demand submission and tracking

Each role gets a distinct dashboard, a role-tailored left navigation, a header role badge, and a personal Profile page describing identity, role, hierarchy level, and capabilities.

The Demand Lifecycle (Stage Machine)

A demand advances through an ordered set of stages. The planner pipeline drives the early stages synchronously; the background worker drives execution.

```
ingested -> understanding -> deciding -> allocating -> awaiting_approval
         -> executing -> monitoring -> explaining -> completed
```

Terminal/auxiliary stages: `failed`, `cancelled`. The sanitizer hides `failed` and `cancelled` demands (and sensitive fields) from the higher-manager view.

End-to-End Flow

1. Client writes a demand

The client submits a free-text description through the client portal (`POST /api/portal/requests`) or an internal user creates one through the wizard (`POST /api/demands`). Minimal input is accepted; the platform infers structure.

2. AI back-and-forth clarification (multi-turn, with options)

If the demand lacks detail, the AI delivery architect asks targeted clarifying questions. Each question includes a short rationale and three suggested answer options, while still allowing a free-text custom answer.

- `POST /api/demands/clarify` returns the first set of questions plus a completeness score.
- `POST /api/demands/converse` continues the conversation, acknowledging the latest answer, asking focused follow-ups, and reporting whether the demand is now detailed enough (`ready_for_plan`).

The engine has an LLM path and a deterministic heuristic fallback, so the conversation works even offline (demo mode).

3. AI produces a detailed plan

On creation, the pipeline runs Understanding -> Reuse detection -> Decision -> Allocation and persists the demand in `awaiting_approval` with a full snapshot:

- Understanding: problem type, domain, complexity, urgency, scope days, required skills, key features, summary.
- Decision: execution mode (AI agent / human team / hybrid / reuse), project type, estimated cost, estimated time, confidence, risk factors.
- Allocation: a team drawn from a 50-person bench (humans, AI agents, partner vendors) with coverage scoring and per-resource match scores.

4. DB-sourced allocation and "should we move this member?"

After allocation, the pipeline cross-references the live `team_members` table. If a recommended person is already committed to another active project, the resource is flagged with a reallocation signal: `move_recommended`, a `move_probability` (skill-fit based), a `move_importance` grade (high/medium/low), and a human-readable `move_rationale`. The manager sees this in the plan and can decide whether to move the person.

5. Reuse suggestions

Reuse detection compares the demand against a library of past projects (pgvector similarity with a keyword fallback). When a strong match exists, the plan shows which components can be reused, why (and why not), components kept versus replaced, and estimated savings in days.

6. Routing and notification

The demand is routed to the unit manager. A durable notification is created (`approval_needed`) so the manager is alerted, in addition to the dashboard's pending-approvals queue. Notifications are also emitted on approval, reassignment, task handoff, and SWON/WON state changes.

7. Manager review

The manager reviews the plan and can:

- Chat with the AI planning copilot (`POST /api/demands/{id}/manager-chat`).
- Chat with the client through the portal thread (`POST /api/portal/requests/{id}/messages` and `.../agent-chat`).
- Share a live progress link with the client by email (`POST /api/demands/{id}/share-link`), which sets the demand's preview URL and records the send in an email outbox (`GET /api/demands/{id}/emails`).

8. Editable team (members, trainers, AI-learners)

Before approving, the manager can edit the team granularly (`PATCH /api/demands/{id}/team`): add or remove members, add trainers (humans upskilling the squad), or add AI-learners (agents shadowing the project for training

data). A catalog of addable resources is available at `GET /api/demands/team/catalog`. Edits are persisted into the allocation, audited, and notified.

9. Approval -> production

The manager approves (`POST /api/demands/{id}/approve`), which moves the demand to `executing` and enqueues the background pipeline.

10. Production: scaffold, auto-publish, commits

The agent fleet scaffolds the project (Vite/React/Tailwind/Supabase template plus generated files), runs through planning, frontend, backend, devops, QA, and docs agents, and stores artifacts. When GitHub auto-publish is configured, the worker pushes the generated project to the remote and records an agent commit. Human contributors record their commits against the demand (`POST /api/demands/{id}/commits`), producing a unified commit timeline.

11. Live tracking and sanitized portfolio

Throughout execution, WebSocket events stream stage and agent progress. Managers track tasks, SWON/WON, blockers, and SLA risk. The higher-manager portfolio is sanitized: failed/cancelled demands and sensitive fields (errors, risk factors, coverage scores) are stripped before they are shown.

Commercial Records: SWON and WON

ForgeOS tracks TCS-style work orders:

- SWON (Service Work Order Number) — the engagement-level record with a lifecycle state, SOW summary, customer LOA reference, and total value.
- WON (Work Order Number) — per-resource billing records under a SWON with allocation percentage, cost centre, and monthly value.

Audit

Every meaningful mutation writes an `AuditEvent` (entity kind, entity id, actor, action, diff, reason). The audit API supports filtering and pagination, and the viewer/manager dashboards surface recent activity.

Key Architectural Components

- Backend: FastAPI app, SQLAlchemy async ORM, Alembic migrations, Arq worker, pgvector for reuse, S3/MinIO artifact storage, smart LLM model router with a provider fallback chain.
- Frontend: React + Vite + Tailwind + React Router, role-aware shell, demand wizard with AI clarification chat, role-specific dashboards and Profile page.
- Auth: dev bypass for local development; Clerk JWT in production. RBAC via `require_role` and per-role route guards.

ForgeOS — REST API Reference

All endpoints are JSON over HTTP. In local/dev mode, auth is bypassed and the caller is the auto-provisioned dev user (default role: manager). In production, a Clerk JWT bearer token is required. Role-gated endpoints return 403 when the caller lacks an allowed role.

Conventions: `{id}` is a public identifier where noted (demands `DMD-XXXX`, SWON `SWON-XXXX`, WON `WON-XXXX`, tasks `TSK-XXXX`) and a UUID otherwise.

Demand & Pipeline (prefix /api)

Method	Path	Role	Purpose
POST	/api/demands/clarify	any	Generate initial clarifying questions (with options)
POST	/api/demands/converse	any	Continue the multi-turn clarification
POST	/api/demands	any	Create a demand; runs the full plan synchronously
POST	/api/demands/{id}/approve	any	Approve and enqueue execution
POST	/api/demands/{id}/manager-chat	any	Manager planning copilot
GET	/api/demands	any	List/paginate/filter/sort demands
GET	/api/demands/{id}	any	Single demand plus agent runs
PATCH	/api/demands/{id}/stage	any	Change stage with reason (audited)
PATCH	/api/demands/{id}/reassign	any	Reassign manager/leader/middleware
POST	/api/demands/{id}/share-link	any	Email a live link to the client
GET	/api/demands/{id}/emails	any	List emails sent for a demand
POST	/api/demands/{id}/commits	any	Record a commit against a demand
GET	/api/demands/{id}/commits	any	List commits for a demand
GET	/api/demands/team/catalog	any	Addable resources (bench, trainers, AI-learners)
PATCH	/api/demands/{id}/team	any	Add/remove team members on a plan

POST /api/demands/clarify

Request: `{ "text": "<demand text>" }` Response: `{ "questions": [ { "id", "question", "why", "category", "options": ["..."] } ], "completeness_score": 0.0-1.0 }`

POST /api/demands/converse

Request: `{ "text": "<demand>", "history": [ { "role": "user|assistant", "content": "" } ], "message": "<latest user message>" }` Response: `{ "message", "follow_up_questions": [ { "id", "question", "why", "category", "options" } ], "ready_for_plan": bool, "completeness_score" }`

POST /api/demands

Request: `{ "text": "<demand>", "clarifications": [ { "question_id", "question", "answer" } ] }` (clarifications optional) Response: `{ "demand_id", "stage": "awaiting_approval", "understanding", "decision", "allocation", "similar_projects", "reuse_score" }` The allocation team entries carry the reallocation signal fields: `kind`, `currently_allocated_to`, `move_recommended`, `move_probability`, `move_importance`, `move_rationale`.

GET /api/demands

Query: `stage`, `search`, `sort`, `order` (asc/desc), `limit`, `offset`. Response: `{ "items": [ ...demand ], "total", "limit", "offset", "has_more" }`

PATCH /api/demands/{id}/stage

Request: `{ "stage": "<new stage>", "reason": "<optional>" }`

PATCH /api/demands/{id}/reassign

Request: `{ "field": "assigned_manager_id|assigned_leader_id|assigned_middleware_id", "user_id": "<uuid>", "reason": "<optional>" }`

POST /api/demands/{id}/share-link

Request: `{ "client_email": "x@y.com", "link": "<optional override>", "message": "<optional>" }` Response: `{ "status": "ok", "preview_url", "email": { "id", "to", "delivered", "provider" } }`

POST /api/demands/{id}/commits

Request: `{ "sha", "author", "message", "files_changed", "branch", "is_agent", "task_public_id"? }`

PATCH /api/demands/{id}/team

Request: `{ "add": [ { "name", "title", "resource_type", "kind": "member|trainer|learner", "skills", "cost_per_day", "allocation_percentage" } ], "remove": ["<name>"], "reason": "<optional>" }` Response: `{ "status": "ok", "allocation": { ...updated allocation } }`

Notifications (prefix /api/notifications)

Method	Path	Role	Purpose
GET	/api/notifications	any	List notifications + unread count
POST	/api/notifications/{id}/read	any	Mark one as read
POST	/api/notifications/read-all	any	Mark all as read

GET response: `{ "items": [ { "id", "kind", "title", "body", "entity_kind", "entity_id", "read", "created_at" } ], "unread_count" }` Notifications are emitted on: demand routing (`approval_needed`), approval (`approved`), reassignment (`assignment`), task handoff (`handoff`), SWON state (`swon_state`), WON state (`won_state`), team edits (`team_edited`).

SWON (prefix /api/swon)

Method	Path	Role	Purpose
GET	/api/swon	any	List SWONs for the tenant
GET	/api/swon/{id}	any	Get a SWON by public id or UUID
POST	/api/swon	manager, higher_manager	Create a SWON
PATCH	/api/swon/{id}/state	manager, higher_manager	Update lifecycle state

Create request: `{ "demand_id": "<uuid>", "customer_loa_ref"?, "sow_summary"?, "total_value_inr"? }`

WON (prefix /api/won)

Method	Path	Role	Purpose
GET	/api/won	any	List (filter by swon, else tenant)
POST	/api/won	manager, higher_manager	Create a WON under a SWON
PATCH	/api/won/{id}	manager, higher_manager	Update WON state

Create request: `{ "swon_id": "<uuid>", "billable", "resource_id"?, "cost_centre"?, "allocation_pct", "monthly_value_inr"? }`

Tasks (prefix /api/tasks)

Method	Path	Role	Purpose
GET	/api/tasks	any	List (by demand_id/owner_id/ status; defaults to caller)
GET	/api/tasks/{id}	any	Get a task
POST	/api/tasks	manager, leader, middleware	Create a task
PATCH	/api/tasks/{id}/status	any	Update status (+ blocked reason)
POST	/api/tasks/{id}/updates	any	Add a comment/update
GET	/api/tasks/{id}/timeline	any	Merged updates + handoffs
POST	/api/tasks/{id}/handoff	any	Request a handoff (notifies assignee)

Audit (prefix /api/audit)

Method	Path	Role	Purpose
GET	/api/audit	any	Paginated activity log

Query: `entity_kind`, `entity_id`, `action`, `actor`, `since`, `limit` (<=500), `offset`.

Dashboards (prefix /api/dashboard)

Method	Path	Role	Purpose
GET	/api/dashboard/executive	executive, higher_manager, manager	Org KPIs
GET	/api/dashboard/manager	manager, higher_manager, executive	Manager console data
GET	/api/dashboard/leader	leader, delivery_team, manager, higher_manager, executive	Team execution data

Reports (prefix /api/reports)

Method	Path	Role	Purpose
GET	/api/reports/delivery	manager, higher_manager	Delivery metrics

Method	Path	Role	Purpose
GET	/api/reports/team-performance	manager, higher_manager	Team performance
GET	/api/reports/demand-aging	manager, higher_manager	Demand aging
GET	/api/reports/sla-compliance	manager, higher_manager	SLA compliance
GET	/api/reports/swon-detail	manager, higher_manager	SWON detail
GET	/api/reports/portfolio	higher_manager, manager	Portfolio (supports sanitized=true)

Most reports accept `format=json|csv|excel` for export.

Client Portal (prefix /api/portal)

Method	Path	Role	Purpose
GET	/api/portal/requests	any	List client requests
POST	/api/portal/requests	any	Create a request; infers and plans
PATCH	/api/portal/requests/{id}	any	Update status/plan/approved team
POST	/api/portal/requests/{id}/messages	any	Add a thread message
POST	/api/portal/requests/{id}/agent-chat	any	AI copilot for the request
GET	/api/portal/team	any	List portal team members
POST	/api/portal/team	any	Create a team member
PUT	/api/portal/team/{id}	any	Update a team member

Create request body: `{ "client": { "name", "email", "company", "role" }, "description", "industry"?, "priority"?, "timeline"?, "budget_range"? }`

GitHub, Projects, Artifacts

Method	Path	Role	Purpose
POST	/api/projects/{id}/github/push	any	Publish generated project to a remote
GET	/api/projects/{id}/files	any	List generated files
GET/PUT	/api/projects/{id}/files/{path}	any	Read/write a file
POST	/api/projects/{id}/server/start	any	Start the dev preview server
GET	/artifacts/{key}	any	Stream a stored artifact

Settings & Health (prefix /api)

Method	Path	Role	Purpose
GET	/api/health	none	Liveness probe (no auth)
GET	/api/settings	any	Read runtime settings
PUT	/api/settings	any	Update runtime settings
GET	/api/llm/routing	any	Current model routing table

Method	Path	Role	Purpose
GET	/api/llm/models	any	Available models

WebSocket

`WS /ws` — per-tenant socket. Sends an `init` payload, relays Redis pub/sub pipeline/agent events, and accepts browser-LLM bridge responses and pings.

ForgeOS — Test Catalog

ForgeOS ships a two-tier end-to-end test suite: backend API tests (pytest + httpx against a real Postgres) and frontend UI tests (Playwright driving a real browser). This catalog documents every test and how to run them.

How to Run

Prerequisites: start the dev infrastructure (Postgres + Redis + MinIO):

```
docker compose -f deploy/docker-compose.dev.yml up -d
```

Backend (99 test cases):

```
cd backend && python3 -m pytest -q
```

Frontend (41 UI test cases):

```
cd frontend && npx playwright test
```

Backend Test Architecture

`backend/tests/conftest.py` configures the environment before importing the app: it points `DATABASE_URL` at a dedicated `forgeos_test` database on the dev Postgres (pgvector image), enables demo mode (deterministic, offline heuristics) and the dev auth bypass. A session-scoped fixture creates the schema and seeds the role catalog; data tables are truncated before each test (roles preserved). A session-scoped event loop keeps the async engine's pooled connections valid.

Fixtures: `client` (httpx AsyncClient over the ASGI app), `db_session` (raw session for assertions/seeding), `dev_identity` (the auto-provisioned dev tenant/user), and `as_role(*slugs)` which assigns roles to the dev user so role-gated routes can be exercised.

Backend Suites

test_smoke.py — harness sanity (4)

- test_health_ok — health endpoint responds.
- test_dev_auth_default_manager — dev user can read settings.
- test_as_role_grants_role — granting a role unlocks the executive dashboard.
- test_create_demand_smoke — creating a demand returns a full plan.

test_health_settings.py — health, settings, routing (6)

- test_health_no_auth_required — /api/health needs no auth.
- test_get_settings — settings include demo_mode true.
- test_update_settings_roundtrip — agent_concurrency persists.
- test_update_settings_clamps_values — worker_max_jobs is clamped.
- test_llm_routing — routing table returns.
- test_llm_models — model list returns.

test_demand_pipeline.py — intake pipeline (15)

- test_clarify_returns_questions_with_options — clarify questions include options.
- test_clarify_detailed_demand_is_more_complete — completeness scales with detail.
- test_converse_returns_followups_and_readiness — converse returns follow-ups + readiness.
- test_converse_becomes_ready_with_rich_history — multi-turn returns a valid score.
- test_create_demand_full_plan — understanding/decision/allocation/reuse present.
- test_create_demand_with_clarifications_enriches_text — answers enrich raw text.
- test_list_demands_pagination_and_total — pagination + total + has_more.
- test_list_demands_filter_by_stage — stage filter.

- test_list_demands_search — text search.
- test_get_demand_includes_agent_runs_key — single demand includes agent_runs.
- test_get_demand_404 — unknown demand returns 404.
- test_stage_change_records_audit — stage change writes an audit event.
- test_reassign_demand — reassign succeeds.
- test_reassign_invalid_field — invalid field returns 400.
- test_manager_chat_fallback — manager copilot returns a response.

test_execution.py — approval + worker pipeline (4)

- test_approve_transitions_to_executing — approve moves to executing + enqueues.
- test_approve_idempotent_when_not_awaiting — repeat approve is safe.
- test_run_full_pipeline_completes — full pipeline reaches completed with runs/artifacts.
- test_run_full_pipeline_handles_executor_failure — executor failure marks failed.

test_rbac.py — role access + sanitizer (10, several parametrized)

- test_executive_dashboard_allowed_roles — executive/higher_manager/manager allowed.
- test_executive_dashboard_forbidden_roles — leader/member/viewer/client forbidden.
- test_manager_dashboard — manager dashboard allowed.
- test_leader_dashboard_allowed — leader dashboard allowed.
- test_leader_dashboard_forbidden_for_viewer — viewer forbidden.
- test_reports_allowed_for_manager — all reports allowed for manager.
- test_reports_forbidden_for_member — reports forbidden for member.
- test_report_csv_export — CSV export works.
- test_portfolio_sanitized_no_failed_or_risk_leak — sanitized portfolio hides failed/risk.
- test_portfolio_unsanitized_allowed_for_manager — unsanitized portfolio allowed.

test_delivery.py — SWON/WON/tasks/audit (7)

- test_swon_create_requires_role — non-manager cannot create a SWON.
- test_swon_lifecycle — SWON create/list/get/state.
- test_won_lifecycle — WON create/list/state.
- test_task_full_lifecycle — task create/status/update/timeline/list.
- test_task_create_forbidden_for_member — member cannot create tasks.
- test_task_handoff — handoff succeeds.
- test_audit_pagination_and_filter — audit pagination + entity filter.

test_portal.py — client portal (5)

- test_portal_create_request_infers_plan — request creates a planned demand.
- test_portal_list_requests — listing works.
- test_portal_add_message_and_patch — messages + status patch.
- test_portal_agent_chat — agent copilot responds.
- test_portal_team_crud — team create/list/update.

test_allocation_move.py — reallocation signal (4)

- test_augment_marks_busy_member_for_move — busy member flagged with probability/importance.
- test_augment_ignores_available_members — available member not flagged.
- test_augment_ignores_agents — AI agents are never moved.
- test_create_demand_surfaces_move_when_humans_busy — busy humans surface the signal end to end.

test_notifications.py — notification feed (6)

- test_demand_creation_emits_approval_notification — approval notification emitted.

- test_mark_one_read_decrements_unread — marking one read decrements the count.
- test_mark_all_read — read-all clears unread.
- test_unread_only_filter — unread filter returns only unread.
- test_reassign_notifies_assignee — reassignment notifies the assignee.
- test_swon_state_change_notifies — SWON state change emits a notification.

test_email_sharelink.py — live-link email (4)

- test_share_link_sets_preview_and_logs_email — preview URL set + email logged.
- test_list_demand_emails — emails are listed for a demand.
- test_share_link_custom_link — a custom link is honored.
- test_share_link_unknown_demand_404 — unknown demand returns 404.

test_team_edit.py — editable team (5)

- test_team_catalog_includes_trainers_and_learners — catalog has members/trainers/learners.
- test_add_trainer_and_learner — adding a trainer and AI-learner works.
- test_remove_member — removing a member works.
- test_team_edit_recomputes_cost_and_audits — cost recomputed + audited.
- test_team_edit_dedupes — duplicate adds are de-duplicated.

test_github_commits.py — publisher + commits (8)

- test_safe_branch_valid / test_safe_branch_invalid — branch validation.
- test_remote_with_token_injects_token / passthrough — remote token handling.
- test_auto_publish_noop_when_unconfigured — no publish when unconfigured.
- test_auto_publish_runs_when_configured — publish + agent commit + event when configured.
- test_commit_create_and_list — commit CRUD.
- test_commit_unknown_demand_404 — unknown demand returns 404.

test_sanitizer.py — sanitizer unit tests (8)

Covers sanitize_demand (failed/cancelled/valid), portfolio exclusion of sensitive stages and fields, no individual user names leaked, audit-event sanitization, and assert_no_leaks catching nested leaks.

Frontend Test Architecture

`frontend/e2e/helpers/auth.ts` seeds a logged-in session directly into localStorage (the exact shape the app writes), bypassing the custom login form. `frontend/e2e/helpers/api.ts` (`mockApi`) intercepts `/api/**` and returns deterministic, correctly-shaped responses so specs do not depend on a running backend. Playwright starts the Vite dev server automatically.

Frontend Specs

delivery-happy-path.spec.ts (6)

Manager console, sanitized higher-manager portfolio (no risk leak), leader board, member work, the dev delivery component gallery, and the reports page.

auth-roles.spec.ts (8 literal, expands per role)

Each role lands on its own distinct dashboard with a persona banner; no-session redirects to login; viewer denied executive; member denied reports; client routed to the client landing; navigation is role-tailored (manager has Settings, viewer does not); header shows the active role badge.

profile.spec.ts (4)

Manager profile shows identity/role and a demands panel; member profile shows the tasks panel and the correct role; viewer profile shows read-only capabilities; the header profile link navigates to /profile.

dashboards.spec.ts (6)

Executive, sanitized higher-manager, manager, middleware, leader, and member dashboards each render their headline.

manager-wizard.spec.ts (2)

Step 1 to the clarify chat shows AI questions with clickable option chips that populate the free-text answer; the clarity progress bar is shown.

client-demand.spec.ts (1)

Client describes an outcome, then sees the AI clarify chat with option chips and a custom answer box.

delivery.spec.ts (2)

The delivery view renders the pipeline, KPIs, and the commit timeline; the share- live-link dialog sends to the client and closes on success.

notifications.spec.ts (2)

The bell shows an unread badge and lists notifications; mark-all-read clears it.

reports-audit.spec.ts (2)

The manager reports page shows export buttons and tabs; the audit history page renders.